

Pet Commerce Platform

Detailed Product, Admin CMS, Inventory, Payments, Analytics & Development Blueprint

Pet Commerce Platform — Master Build Specification

1. Project Goal

Build a production-ready, full-stack pet-products e-commerce website using **Next.js** as the application framework and **Supabase** as the backend platform.

The business sells pet food/feed, toys, accessories and potentially other pet-related products. The system must be configurable from an admin CMS so the owner can manage the catalogue and operations without changing code.

Core principles

- One Next.js application: storefront + admin + server-side business logic.
- Supabase: PostgreSQL database, Authentication, Storage and Row Level Security.
- Vercel: deployment for the Next.js application.
- Keep the initial operating cost as close to zero as practical using free tiers.
- Do not introduce NestJS or another backend framework unless a later scaling requirement justifies it.
- Avoid unnecessary libraries.
- Prefer native Next.js/React/browser capabilities over dependencies.
- The storefront should be friendly and premium, not cartoonish.
- Animations should be subtle, purposeful and lightweight.
- Admin UI should prioritize speed, clarity and usability over decorative animation.
- Build in phases. **Do not implement all phases at once.**
- After each phase, verify the work, fix issues, then wait for approval before moving to the next phase.

2. Recommended Stack

Required

- Next.js (App Router, TypeScript; use the current stable version appropriate for the project)
- React
- Tailwind CSS
- Supabase
 - PostgreSQL
 - Auth
 - Storage
 - Row Level Security

- Vercel
- Razorpay for Indian payments
- Resend for transactional email

Minimal supporting libraries

Use only libraries that solve a real problem.

Recommended:

- @supabase/ssr for Supabase authentication/session handling
- zod for server-side input validation
- lucide-react for lightweight icons
- recharts only where dashboard charts genuinely require it
- A small drag-and-drop solution only if native HTML5 drag/drop is insufficient for product ordering

Do not add a component library, animation framework, state-management library or form library merely for convenience. Use React/Next.js primitives where practical.

3. Product Experience

Storefront

Required pages:

- Home
- Shop
- Category
- Product detail
- Search results
- Cart
- Checkout
- Order success
- Customer account
- Orders
- Order detail
- Login/register
- Contact
- About
- Privacy Policy
- Terms
- Shipping Policy
- Returns/Refund Policy

Catalogue hierarchy

The catalogue must be data-driven.

Recommended model:

Pet Type → Category → Subcategory → Product

Examples:

- Dog
 - Food
 - Toys
 - Accessories
- Cat
 - Food
 - Toys
 - Accessories
- Bird
 - Food
 - Accessories

Do not hardcode these values into UI logic.

The admin must be able to create, edit, reorder, archive and delete catalogue entities where safe.

A product may belong to one or more relevant pet types/categories where the business model requires it.

4. Product Model

Each product should support:

- Name
- Slug
- SKU
- Short description
- Full description
- Pet type
- Category
- Subcategory
- Brand
- Product images
- Thumbnail
- MRP
- Selling price

- Discount
- Purchase/cost price
- Tax/GST fields as required by the business
- Stock quantity
- Low-stock threshold
- Published/unpublished status
- Featured status
- Product ordering/sort position
- SEO title
- SEO description
- SEO keywords/metadata where appropriate
- Created/updated timestamps

Variants

Support variants where useful, for example:

- 500g
- 1kg
- 5kg
- Small / Medium / Large
- Colour

Each variant should be able to have its own:

- SKU
- price
- cost price
- stock
- optional MRP
- optional image

Do not force variants onto products that do not need them.

5. Inventory — Online + Offline Sales

Inventory is a core business requirement.

The system must support both:

1. Online orders placed through the website.
2. Offline sales recorded manually by the admin.

Example:

Starting stock = 1,000

Online sales = 100

Available stock = 900

Owner sells 200 units offline.

Available stock must become 700.

Never rely only on a single editable stock number

Maintain an inventory movement/history system.

Inventory movements should record:

- Product/variant
- Quantity change
- Movement type
- Reason
- Reference order where applicable
- Previous quantity
- New quantity
- User/admin who made the change
- Timestamp
- Optional note

Movement types can include:

- ONLINE_SALE
- OFFLINE_SALE
- STOCK_RECEIVED
- STOCK_ADJUSTMENT
- RETURN
- DAMAGED
- CANCELLED_ORDER
- RESTOCK

Offline sale UI

Admin should have a simple action:

Record Offline Sale

Fields:

- Product/variant
- Quantity
- Sale price if business wants offline revenue reporting

- Optional customer/reference
- Note

Submitting it should safely reduce stock and create an inventory movement.

Stock safety

Do not allow accidental negative inventory unless an explicit business setting permits it.

Inventory-changing operations should be atomic and protected against race conditions.

6. Orders

Order lifecycle:

- Pending payment
- Confirmed
- Processing
- Packed
- Shipped
- Out for delivery
- Delivered
- Cancelled
- Return requested
- Returned
- Refunded

Do not make statuses purely visual. Store them in the database.

Every order should contain:

- Order number
- Customer
- Items
- Product/variant snapshot
- Quantity
- Price at purchase
- Discount
- Tax
- Shipping charge
- Total
- Payment status
- Payment provider/reference ID
- Fulfilment status

- Shipping address snapshot
- Timestamps

Important

Order items should preserve the price/product information at the time of purchase. If an admin changes a product price later, old orders must not change.

7. Payments

Use Razorpay for initial Indian payment integration.

Flow:

1. Customer creates checkout.
2. Server validates cart and current prices/stock.
3. Server creates payment/order request.
4. Customer completes payment.
5. Server verifies payment signature/status.
6. Only after verified payment should the order become confirmed.
7. Inventory should be reduced exactly once.
8. Send confirmation email.

Never trust client-submitted totals.

The server must calculate:

- item subtotal
- discount
- tax
- shipping
- grand total

from trusted database values.

8. Shipping

Keep shipping logic configurable.

Admin should be able to configure:

- Shipping charge
- Free-shipping threshold
- Serviceable areas/pincodes if needed

- Delivery-related notes

Design the system so a shipping provider API can be added later without rewriting checkout.

For the first version, a simple configurable shipping model is enough unless the business already has a courier API requirement.

9. Discounts and Coupons

Support:

- Percentage discount
- Fixed discount
- Minimum order value
- Start/end date
- Usage limit
- Per-customer usage limit
- Active/inactive state
- Optional product/category applicability

Validate coupons on the server.

Do not trust discount values from the browser.

10. Admin CMS

Admin panel can be an SPA-like experience inside the Next.js application.

Admin sections

- Dashboard
- Products
- Categories
- Pet Types
- Subcategories
- Brands
- Inventory
- Orders
- Customers
- Coupons
- Homepage/content sections
- Reviews
- Reports

- Settings
- Audit log

Product management

Admin can:

- Add product
- Edit product
- Duplicate product
- Publish/unpublish
- Archive
- Delete where safe
- Upload/reorder images
- Manage variants
- Update prices
- Update cost
- Update stock
- Mark featured
- Change display order

Product ordering

The owner must be able to move products forward/backward in the storefront ordering.

Use an explicit `sort_order/position` field.

Do not depend on database insertion order.

The same concept can be used for categories and homepage sections.

11. Dashboard and Analytics

Dashboard should immediately answer:

- Today's sales
- Today's orders
- Total revenue
- Gross sales
- Discounts given
- Shipping collected
- Taxes collected
- Estimated cost of goods
- Estimated gross profit

- Low-stock products
- Out-of-stock products
- Top-selling products
- Recent orders
- Online vs offline sales

Cost/profit reporting

Store purchase/cost price separately from selling price.

At minimum calculate:

Gross Revenue = product selling value before discounts as defined by the business

Net Sales = revenue after discounts

Estimated Gross Profit = Net Product Sales – Cost of Goods Sold

Do not call this net profit because operating expenses, payment fees, shipping costs, salaries, etc. may not be included.

Reporting periods

Support:

- Today
- Yesterday
- Last 7 days
- Last 30 days
- This month
- Custom date range

Admin reports should distinguish:

- Online sales
- Offline sales
- Combined sales

12. Audit Log

Important admin actions should be logged.

Examples:

- Product created
- Product edited
- Product deleted/archived
- Price changed

- Cost changed
- Stock changed
- Offline sale recorded
- Order status changed
- Coupon created/changed
- Settings changed

Store:

- actor/admin
- action
- entity
- entity ID
- previous value where appropriate
- new value where appropriate
- timestamp

This protects the business when someone asks, “Who changed the stock?”

13. Authentication and Authorization

Use Supabase Auth.

Roles should be extensible, for example:

- ADMIN
- STAFF

Start with one admin role if necessary, but design the database so permissions can expand later.

Protect admin routes server-side.

Never rely only on hiding admin links in the UI.

Use Supabase Row Level Security.

Service-role credentials must never be exposed to the browser.

14. Database Design

Core tables should include approximately:

- profiles
- roles / admin_roles if required
- pet_types

- categories
- subcategories
- brands
- products
- product_variants
- product_images
- inventory_movements
- carts
- cart_items
- orders
- order_items
- payments
- addresses
- coupons
- coupon_redemptions
- reviews
- homepage_sections
- audit_logs
- site_settings

Use foreign keys, indexes, unique constraints and timestamps.

Use soft deletion/archive where historical relationships make hard deletion dangerous.

15. Supabase Storage

Use Supabase Storage for product images and other media.

Requirements:

- Store optimized images.
- Use predictable storage paths.
- Validate uploaded file type/size.
- Generate appropriate thumbnails where practical.
- Do not store huge original images unnecessarily.
- Keep product image ordering in the database.

16. SEO

Implement:

- Metadata per page

- Product metadata
- Category metadata
- Canonical URLs
- Sitemap
- Robots configuration
- Open Graph metadata
- Product structured data where appropriate
- Clean slugs
- Good semantic HTML

Product pages must be indexable.

17. UI / Visual Direction

Theme:

Modern pet retail — warm, clean, trustworthy, friendly, premium.

Avoid:

- cartoon-heavy illustrations
- childish UI
- excessive rounded cards
- excessive gradients
- excessive shadows
- noisy animations
- huge decorative graphics
- too many fonts
- unnecessary visual effects

Use:

- one primary typeface family
- strong typography hierarchy
- generous whitespace
- high-quality product photography
- restrained accent colour
- subtle rounded corners
- clean cards
- clear CTAs

The final logo is not decided yet. Keep branding tokens centralized so logo/colors can be replaced later without redesigning the application.

Animation

Use minimal motion:

- product image hover
- subtle card transitions
- button feedback
- page/section reveal where useful
- cart feedback
- skeleton loading

Animations must respect `prefers-reduced-motion`.

Do not install a heavy animation library unless native CSS/React transitions cannot achieve the required result.

18. Performance

Targets:

- Fast first load
- Optimized images
- Server Components by default
- Client Components only when interaction requires them
- Avoid unnecessary global state
- Avoid unnecessary API requests
- Use pagination for large admin tables
- Use database indexes
- Use caching/revalidation intentionally
- Lazy-load non-critical UI
- Keep JavaScript bundles small

The admin should remain fast even with thousands of products/orders.

19. Security Rules

Never:

- expose Supabase service-role key
- trust client totals
- trust client discount calculations
- allow arbitrary admin access
- modify stock directly from untrusted client code
- expose private customer data unnecessarily

Validate all server inputs with schemas.

Use authorization checks on every admin mutation.

Protect payment verification.

Use RLS policies correctly.

20. Suggested Project Structure

```
src/  
app/  
(store)/  
page.tsx  
shop/  
category/  
product/  
cart/  
checkout/  
account/  
admin/  
page.tsx  
products/  
categories/  
inventory/  
orders/  
customers/  
coupons/  
reports/  
settings/  
api/  
payments/  
webhooks/  
components/  
ui/  
store/  
admin/  
features/  
products/  
inventory/  
orders/
```

payments/
coupons/
analytics/
lib/
supabase/
validation/
pricing/
permissions/
utils/
types/
config/
styles/
supabase/
migrations/
seed/

Exact structure may evolve during implementation, but keep business logic out of random UI components.

21. Development Phases

Phase 0 — Project Rules and Planning

Before writing application code:

- Inspect the repository.
- Confirm Next.js/React/TypeScript setup.
- Confirm environment variable strategy.
- Establish linting/formatting.
- Establish Tailwind and design tokens.
- Create this project structure.
- Do not build business features yet.

Stop after Phase 0 and verify.

Phase 1 — Supabase Foundation

Build:

- Supabase project connection
- Environment variables
- Database migrations

- Auth foundation
- Storage buckets
- RLS foundation
- Base tables
- Seed data for development

Do not build the full storefront yet.

Stop and verify database relationships and RLS.

Phase 2 — Admin Authentication + Shell

Build:

- Admin login
- Protected admin routes
- Admin layout
- Sidebar/navigation
- Header
- Responsive layout
- Basic dashboard shell

Admin should be functional before catalogue management is added.

Stop and verify authorization.

Phase 3 — Catalogue CMS

Build:

- Pet types
- Categories
- Subcategories
- Brands
- Products
- Product images
- Variants
- Product publishing
- Product ordering
- Search/filter/pagination

Stop and test full CRUD.

Phase 4 — Inventory

Build:

- Stock display
- Stock adjustment
- Offline sale recording
- Inventory movement history
- Low-stock threshold
- Out-of-stock state
- Online order stock deduction foundation
- Inventory safety checks

Stop and test scenarios such as 1000 → online 100 → offline 200 = 700.

Phase 5 — Storefront

Build:

- Home
- Shop
- Category pages
- Search
- Product page
- Filters
- Sorting
- Product variants
- Cart

Use real Supabase data.

Stop and test mobile + desktop.

Phase 6 — Checkout + Orders

Build:

- Address collection
- Shipping calculation
- Order creation

- Order history
- Order detail
- Order status
- Cancellation rules

Server calculates every total.

Stop and test failed/duplicate checkout scenarios.

Phase 7 — Razorpay

Build:

- Payment order creation
- Checkout integration
- Signature/status verification
- Payment records
- Successful payment → confirmed order
- Failure handling
- Duplicate callback/webhook protection

Stop and test payment edge cases.

Phase 8 — Email

Use Resend for:

- Order confirmation
- Payment confirmation
- Order status updates
- Cancellation/refund messages where appropriate
- Admin notification for new orders if required

Keep email templates reusable.

Phase 9 — Discounts + Shipping

Build:

- Coupon management
- Coupon validation

- Shipping settings
- Free-shipping threshold
- Server-side calculation

Phase 10 — Analytics + Reports

Build dashboard metrics:

- revenue
- orders
- discounts
- cost of goods
- estimated gross profit
- online/offline split
- top products
- low stock
- date filters

Charts should be limited to useful visualizations.

Do not create dashboards full of meaningless graphs.

Phase 11 — Content + SEO

Build admin controls for:

- Homepage sections
- Featured products
- Banners/content blocks
- SEO fields

Then implement storefront SEO.

Phase 12 — Audit + Hardening

Build:

- audit logs
- permission checks
- error handling
- loading states

- empty states
- validation messages
- rate/abuse protection where appropriate
- security review
- database indexes
- backup/recovery considerations

Phase 13 — Testing

Test:

Storefront

- browsing
- search
- filtering
- product variants
- cart
- checkout
- mobile

Admin

- CRUD
- ordering
- inventory
- offline sale
- order management
- coupons
- analytics

Critical business logic

- stock cannot unexpectedly go negative
- price changes do not change historical orders
- payment cannot create duplicate orders
- payment failure does not confirm an order
- coupon cannot be abused
- unauthorized users cannot access admin data

Phase 14 — Production

Before deployment:

- Production Supabase project
- Production environment variables
- Razorpay production keys
- Resend production configuration
- Domain
- Vercel deployment
- Database migrations
- RLS verification
- Payment webhook configuration
- SEO verification
- Error monitoring
- Final mobile/desktop QA

22. Cursor AI Operating Rules

Cursor must follow these rules:

1. Read this entire specification before changing code.
2. Work only on the current phase.
3. Do not silently implement future phases.
4. Before coding a phase, briefly explain what files will change and why.
5. If a requirement is ambiguous, ask before making a business-critical assumption.
6. Prefer existing project utilities over adding new dependencies.
7. Do not introduce duplicate abstractions.
8. Keep components focused.
9. Keep business logic reusable and testable.
10. Keep database access organized.
11. Never expose secrets.
12. Never bypass RLS as a shortcut.
13. After implementation, run appropriate checks.
14. Fix errors before declaring the phase complete.
15. Summarize completed work and remaining work.
16. Then stop and wait for approval.

Important

Do not give Cursor the instruction to build the entire application in one response.

Use this file as the source of truth and execute one phase at a time.

23. Definition of Done

The application is ready for launch only when:

- Customer can browse products.
- Customer can search/filter.
- Customer can select variants.
- Customer can add to cart.
- Customer can checkout.
- Payment works and is verified server-side.
- Orders are stored correctly.
- Inventory is updated correctly.
- Admin can manage products/categories/pet types.
- Admin can reorder products.
- Admin can record offline sales.
- Admin can see inventory history.
- Admin can see online + offline sales.
- Admin can see revenue, discounts, costs and estimated gross profit.
- Admin can manage orders.
- Admin can manage coupons/shipping settings.
- Admin actions are audited.
- Security rules are verified.
- Mobile and desktop UX are acceptable.
- Performance is acceptable.
- Production environment is configured.

24. Future Scaling — Do Not Build Now

Do not build these unless the business requires them:

- Separate NestJS backend
- Microservices
- Redis
- Elasticsearch
- Complex event-driven architecture
- Native mobile app
- Multi-vendor marketplace
- Advanced warehouse management
- Multiple warehouses
- AI recommendations
- Complex loyalty system

The architecture should make these possible later without forcing them into version one.

Launch a clean, reliable system first. Scale when real usage justifies it.